JavaScript ArrayBuffer

Back to JS page

Table of Contents

ArrayBuffer

An `ArrayBuffer` is a fixed-length, raw binary data buffer in memory. It cannot be read or written directly. Instead, you create a **typed array view** or a **DataView** to read/write the buffer.

Creating an ArrayBuffer

You create an `ArrayBuffer` by specifying its size in bytes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating an ArrayBuffer</h4>
<p id="demo"></p>

<script>
// Create an ArrayBuffer with 16 bytes
const buffer = new ArrayBuffer(16);

let text = "ArrayBuffer created with 16 bytes<br>";
text += "byteLength: " + buffer.byteLength + "<br>";
text += "Max byteLength: " + ArrayBuffer.maxByteLength + "<br>";
text += "byteLength is read-only? " + (buffer.byteLength === 16);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result View Example

Using an Uint8Array with ArrayBuffer

To access an `ArrayBuffer`, you need a typed array view. `Uint8Array` treats each byte as an unsigned 8-bit integer:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using an Uint8Array with ArrayBuffer</h4>
<p id="demo"></p>

<script>
// Create buffer and view
const buffer = new ArrayBuffer(8);
const view = new Uint8Array(buffer);

// Write values
view[0] = 65;   // 'A'
view[1] = 66;   // 'B'
view[2] = 67;   // 'C'
view[3] = 97;   // 'a'
view[4] = 98;   // 'b'

// Read values
let text = "Uint8Array view of ArrayBuffer(8):<br>";
for (let i = 0; i < view.length; i++) {
  text += "view[" + i + "] = " + view[i];
  if (view[i] >= 65 && view[i] <= 90) {
    text += " (char: " + String.fromCharCode(view[i]) + ")";
  }
  text += "<br>";
}
text += "<br>byteLength: " + buffer.byteLength;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result View Example

Using a DataView

A `DataView` provides a flexible interface to read/write different data types at any byte offset:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using a DataView</h4>
<p id="demo"></p>

<script>
const buffer = new ArrayBuffer(16);
const dataView = new DataView(buffer);

// Write different data types
dataView.setInt32(0, 42);        // 4 bytes at offset 0
dataView.setFloat32(4, 3.14);    // 4 bytes at offset 4
dataView.setInt16(8, 255);       // 2 bytes at offset 8

// Read back
let text = "DataView on ArrayBuffer(16):<br>";
text += "getInt32(0): " + dataView.getInt32(0) + "<br>";
text += "getFloat32(4): " + dataView.getFloat32(4) + "<br>";
text += "getInt16(8): " + dataView.getInt16(8) + "<br><br>";

// Check byte order (endianness)
const testBuffer = new ArrayBuffer(2);
const testView = new DataView(testBuffer);
testView.setInt16(0, 256);
text += "Endianness test: ";
if (testView.getUint8(0) === 1) {
  text += "Big-endian";
} else {
  text += "Little-endian";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result View Example

ArrayBuffer.slice()

You can create a new `ArrayBuffer` from a portion of an existing buffer using `slice()`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>ArrayBuffer.slice()</h4>
<p id="demo"></p>

<script>
const buffer = new ArrayBuffer(16);
const view = new Uint8Array(buffer);

// Fill with values
for (let i = 0; i < view.length; i++) {
  view[i] = i * 10;
}

// Slice a portion
const sliced = buffer.slice(4, 12);
const slicedView = new Uint8Array(sliced);

let text = "Original buffer: ";
for (let i = 0; i < view.length; i++) text += view[i] + " ";
text += "<br>";

text += "Sliced buffer (bytes 4-12): ";
for (let i = 0; i < slicedView.length; i++) text += slicedView[i] + " ";
text += "<br><br>";

text += "Original byteLength: " + buffer.byteLength + "<br>";
text += "Sliced byteLength: " + sliced.byteLength;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result View Example

SharedArrayBuffer

A `SharedArrayBuffer` is similar to `ArrayBuffer` but can be shared between workers (threads):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript ArrayBuffer</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>SharedArrayBuffer</h4>
<p id="demo"></p>

<script>
// Create a SharedArrayBuffer
const sharedBuffer = new SharedArrayBuffer(16);
const sharedView = new Int32Array(sharedBuffer);

// Write values
for (let i = 0; i < sharedView.length; i++) {
  sharedView[i] = i * 100;
}

let text = "SharedArrayBuffer created with 16 bytes<br>";
text += "byteLength: " + sharedBuffer.byteLength + "<br>";
text += "Int32Array length: " + sharedView.length + " (4 bytes each)<br><br>";

text += "Values: ";
for (let i = 0; i < sharedView.length; i++) {
  text += sharedView[i] + " ";
}
text += "<br><br>";

text += "SharedArrayBuffer can be used with:<br>";
text += "- Atomics.add()<br>";
text += "- Atomics.load()<br>";
text += "- Atomics.store()<br>";
text += "- Web Workers for multi-threading";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result View Example

Document

Document in project

You can Download PDF file.

Reference

- W3Schools JavaScript ArrayBuffer